
ENGINEERING · FRAUD DETECTION · STREAM PROCESSING**Architecting a 100M Events/Day Fraud Detection Pipeline: Tradeoffs & Lessons**

How we redesigned Western Union's fraud infrastructure from a batch-oriented system to a sub-second streaming pipeline — the architectural decisions, failure modes, and what we would do differently.

Utkarsh Mehrotra · Chief Engineer, Samsung Electronics · Ex-Western Union

100M+

EVENTS PER DAY

<1s

END-TO-END P99

~1,150EVENTS/SECOND
SUSTAINED

Background

Before this system existed, Western Union's fraud analytics team operated under a structural constraint that no amount of hiring or process improvement could fix: the minimum time between a new fraud pattern being identified and a rule being live in production was measured in weeks. A coordinated mule-network attack discovered on a Monday — where funds move through chains of accounts before exiting the system — would continue operating unimpeded until the next sprint boundary, at

minimum. During active fraud campaigns, that exposure window translated directly into unrecoverable losses.

The root cause was architectural. Risk scoring ran on a pull-based batch cycle against a transaction database. By the time a pattern surfaced, the funds had settled and the counterparty accounts were empty. The fraud team had deep expertise in pattern recognition; they had no mechanism to act on that expertise in time. Every intervention was remediation. The business question that drove this project was not "how do we improve latency" — it was "how do we give the fraud team a system that operates at the speed of attacks rather than the speed of sprint cycles."

This post describes the streaming pipeline we built to answer that question: the architecture, the specific decisions that shaped it, the second-order failure modes we did not anticipate, and what we would change.

System Requirements

Before discussing the architecture, it is worth being explicit about the constraints that shaped every decision:

- R1 **Pre-settlement intervention.** Fraud scoring must complete and produce an actionable signal before the transaction settles. Post-settlement flagging is operationally valuable but not the primary goal of this system.

- R2 **Sub-second end-to-end latency.** P99 latency across the full pipeline — ingest through action — must remain under one second at sustained load.

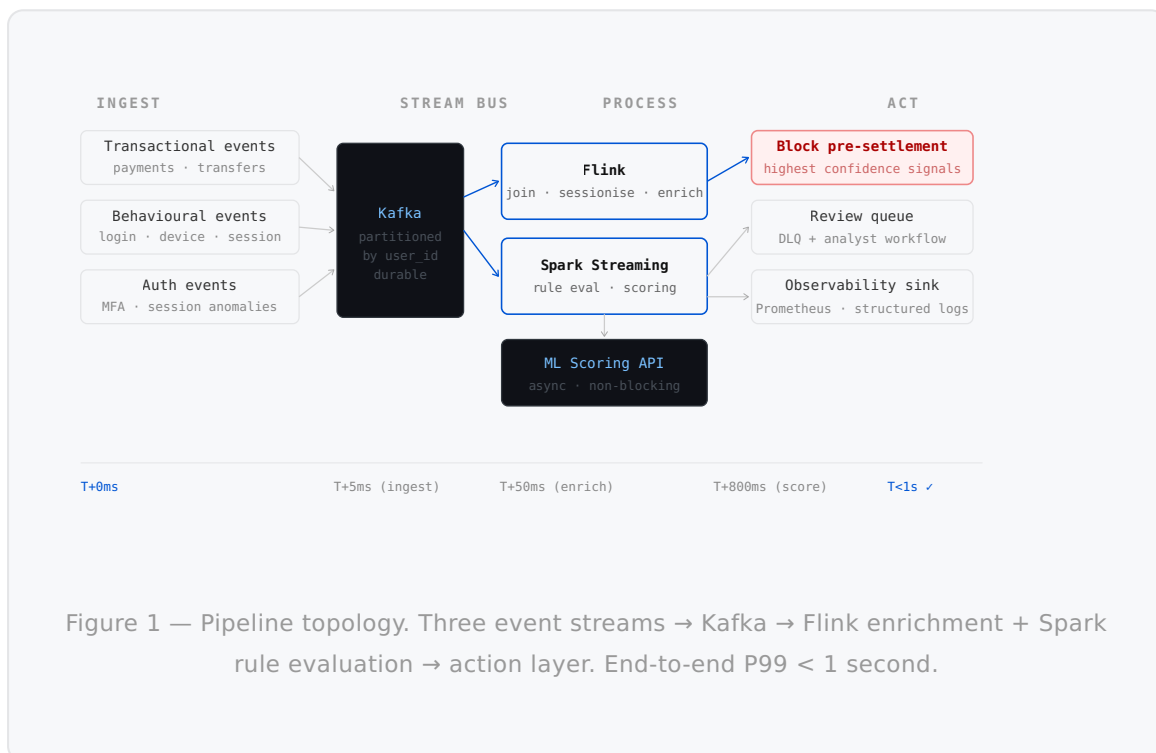
- R3 **No event loss under failure.** Dropping a transactional event is not a data quality issue in this domain — it is a compliance failure. The system must guarantee event durability under partial failures.

R4

Cross-stream correlation. A payment event in isolation is often insufficient to make a reliable fraud determination. The system must correlate transactional signals with behavioral and authentication events for the same user within a time window.

Architecture Overview

The pipeline is structured as three distinct layers: ingest, process, and act. Each layer has a clear ownership boundary and a defined latency budget. The diagram below shows the full topology.



Key Design Decisions

EVENT BUS: KAFKA WITH ENTITY-KEYED PARTITIONING

We evaluated three approaches to event ingestion before landing on Kafka. The first was direct consumption: Flink subscribes to upstream REST webhooks from the transaction processing system. The second was

a managed queue service (AWS SQS). The third was Kafka as a durable log.

Direct webhook consumption was rejected immediately: REST webhooks provide no durability guarantee. When a downstream Flink job fails and restarts during a traffic spike — which is not a hypothetical — events emitted during that window are permanently lost. In a financial fraud system, lost events are a compliance failure, not a data quality issue. SQS was a closer call: it provides durability and at-least-once delivery, but its lack of consumer-side offset management means you cannot replay from a specific point in the event history, and its maximum retention window of 14 days is insufficient for the audit requirements of this domain. Kafka's durable, replayable, offset-managed log was the only option that satisfied all three non-negotiables: durability, replayability, and consumer-controlled positioning.

We partition by `user_id` rather than using default round-robin assignment. This ensures that all events for a given user are processed by the same Flink task manager, making stateful session-level joins possible without cross-partition coordination — which at our throughput would be prohibitively expensive.

PROCESSING: FLINK FOR ENRICHMENT, SPARK FOR RULE EVALUATION

Three processing architectures were evaluated. The first was a unified Flink pipeline handling both enrichment and rule evaluation. The second was Kafka Streams for enrichment with a separate rule execution service. The third was the Flink-plus-Spark split we ultimately deployed.

The unified Flink approach was appealing from an operational simplicity standpoint — one runtime, one deployment model, one set of runbooks. We built a prototype and benchmarked it. Flink's Table API and SQL layer, while mature, added 40–60ms of overhead to complex analytical rule evaluation that Spark's native execution engine handled in under 10ms. At our latency budget, that gap was not acceptable. Kafka Streams was eliminated early: its stateful join primitives are weaker than Flink's, and

its rule evaluation story is essentially non-existent without significant custom development.

The Flink-plus-Spark split carries real operational cost — two runtimes, two deployment pipelines, two monitoring stacks. That cost is worth it because the performance profiles are genuinely distinct: Flink's credit-based flow control and RocksDB state backend are purpose-built for low-latency stateful streaming; Spark's columnar execution engine and SQL optimiser are purpose-built for analytical batch-like workloads. Using one engine for both means accepting its worst characteristics across half the pipeline.

ML INTEGRATION: ASYNCHRONOUS SCORING WITH DETERMINISTIC FALLBACK

ML scoring is integrated as an asynchronous side-channel from the Spark rule evaluation stage. Making this call synchronous would have added 200–400ms of latency per event — consuming most of the available budget. Instead, we issue the ML scoring request asynchronously and merge the result with rule-engine output before emitting the final risk signal.

Critically, the system does not depend on the ML service being available. If the scoring service degrades or exceeds its timeout threshold, the pipeline falls back to rule-only scoring. This was not added as an afterthought — it was a day-one design requirement. Any latency-sensitive path that introduces a hard dependency on an external service is a production incident waiting to happen.

The architectural constraint that shaped every decision: fraud signals must be produced and acted on before settlement completes, not after. Systems that score post-settlement are forensic tools. This system is a prevention mechanism. Losing sight of this distinction leads to building something technically impressive that solves the wrong problem.

Engineering Challenges

BACKPRESSURE MANAGEMENT AT SUSTAINED THROUGHPUT

At 100M events/day the pipeline sustains approximately 1,150 events per second under normal conditions, with spikes of 3–4× that rate around monthly pay cycles and peak international remittance windows. At this throughput, slowdowns in downstream stages — the ML scoring service, the action layer — do not just add latency. They cascade upstream through the enrichment layer into Kafka consumer group lag within seconds, and from there cause Flink task managers to exhaust heap within minutes. Without active backpressure control, a 10-second ML scoring degradation produces a pipeline recovery time measured in hours, not minutes.

We implemented Flink's credit-based flow control with dynamic parallelism scaling, driven by consumer lag metrics exported to Prometheus. Task parallelism scales up automatically when consumer lag on any partition crosses a threshold, and scales down during off-peak windows to reduce infrastructure cost. The critical tuning constraint was that backpressure should slow ingestion rather than drop events. Shedding load by dropping events is not an acceptable degradation mode for a financial fraud pipeline — every dropped event is a transaction that escapes scoring entirely.

EXACTLY-ONCE SEMANTICS: A DELIBERATE TRADEOFF

We evaluated Flink's exactly-once processing mode, which uses two-phase commit between Flink checkpoints and Kafka producers to guarantee that each event is processed precisely once end-to-end. After benchmarking at our target throughput, we chose at-least-once delivery with idempotent consumers instead.

Exactly-once mode at 1,150 events/second introduced approximately 18% additional end-to-end latency from two-phase commit coordination — the checkpoint barrier propagation and producer flush synchronisation

add overhead that compounds at high throughput. At lower throughput, this overhead is absorbed by available headroom. At our sustained rate, it consumed a third of the latency budget before any enrichment or scoring had occurred. More importantly, the action layer was already designed to handle duplicates safely — making the coordination cost of exactly-once a pure overhead with no correctness benefit at the application layer.

```
// Idempotent signal handler — correct under at-least-once delivery
fun handleFraudSignal(event: FraudSignal) {
    val key = "fraud:${event.txnId}:${event.signalType}"
    if (redis.setNX(key, event.score, ttl = 24.hours)) {
        // First time we have seen this signal — act on it
        actionRouter.dispatch(event)
    }
    // Duplicate delivery — already processed, discard
}
```

Blocking a transaction twice is a no-op at the payment rail level. Inserting a duplicate into the analyst review queue is deduplicated at read time. The at-least-once + idempotent consumer pattern eliminates the coordination overhead of exactly-once while maintaining correctness at the application layer.

DECOUPLING RULE UPDATES FROM THE DEPLOYMENT CYCLE

Early in the project, fraud analysts were identifying new attack patterns faster than engineering could ship rule updates. A mule-account pattern detected on Monday required a rule live by Tuesday — not at the next sprint boundary. Treating this as a process problem would be the wrong response; it is an architectural problem.

We externalised rule definitions to a database, hot-loaded by the Spark rule engine on a configurable polling interval. Adding or modifying a rule became a database write owned directly by the fraud analytics team, with no engineering deployment required. Time-to-detection for new fraud patterns dropped from multiple days to under one hour. This single

change produced more measurable business impact than any other technical decision in the project.

Summary of Key Decisions

DECISION	OPTION CHOSEN	RATIONALE
Event bus	Kafka (durable log)	Replay guarantee under failure; REST webhooks provide no durability
Partition key	user_id	Enables stateful per-user joins without cross-partition coordination
Processing split	Flink + Spark	Flink for low-latency stateful joins; Spark for complex rule evaluation
ML integration	Async with fallback	Synchronous call consumes 200–400ms of latency budget
Delivery semantics	At-least-once + idempotent consumers	Exactly-once adds ~18% latency overhead from 2PC at 1,150 events/s; action layer idempotency provides correctness without coordination cost
Rule management	Database-driven hot reload	Removes engineering from the detection-to-rule deployment loop

A Non-Obvious Production Problem: Consumer Group Rebalancing Storms

The most operationally complex failure mode we debugged in production was one that does not appear in any Flink or Kafka tutorial: consumer group rebalancing storms triggered by task manager restarts under sustained load of 1,150 events/second.

The sequence was repeatable. A Flink task manager would restart — due to a garbage collection pause, a transient OOM, or a routine deployment — and trigger a Kafka consumer group rebalance. During rebalance, all partition consumption pauses. Under normal conditions this lasts 3–5 seconds. Under sustained load, with consumer lag already accumulating from the restart, the rebalance triggered a secondary condition: Kafka's `max.poll.interval.ms` was set to its default of 300 seconds, but the time required to process the backlog of accumulated events after the rebalance exceeded that limit. The consumer was marked dead by the group coordinator, triggering another rebalance. The second rebalance caused more lag. The cycle repeated.

The fix required tuning three interdependent parameters together rather than in isolation:

```
# Kafka consumer configuration – tuned for streaming fraud pipeline

# How long the coordinator waits for a consumer to poll before
# declaring it dead. Must exceed max processing time for accumulated lag.
max.poll.interval.ms = 900000    # 15 minutes

# Heartbeat interval must be 1/3 of session timeout per Kafka docs.
# Session timeout controls how quickly the coordinator detects a dead consumer.
session.timeout.ms    = 45000    # 45 seconds
heartbeat.interval.ms = 15000    # 15 seconds

# Limit records fetched per poll to bound per-poll processing time.
```

```
# Prevents a single large batch from exhausting the poll interval.  
max.poll.records      = 200          # down from default 500
```

The non-obvious insight is that `max.poll.interval.ms` and `max.poll.records` must be tuned in relation to each other: if the time to process `max.poll.records` records can exceed `max.poll.interval.ms` under backlog conditions, you will rebalance. We derived the safe value for `max.poll.records` by measuring P99 per-record processing time under peak load, multiplying by the target batch size, and setting `max.poll.interval.ms` to 3× that value to provide headroom for GC pauses. Once these parameters were co-tuned, the rebalancing storm behaviour disappeared entirely.

This class of problem — emergent from the interaction between two independently reasonable configuration defaults — is the kind of operational knowledge that only comes from running a system under real load for long enough to observe it. We documented the tuning rationale explicitly in the runbook so that future engineers would understand not just what the values are, but why they are what they are.

-
- 01 **Observability from day one.** The Prometheus metrics layer and structured logging were added in the final weeks of the project. A streaming system at this scale cannot be tuned, debugged, or operated without latency histograms and consumer lag visibility in place from the start. Production readiness is not a milestone — it is a precondition.

 - 02 **Evaluate Flink-only for rule evaluation.** The Flink/Spark split is architecturally sound but operationally expensive: two runtimes, two deployment pipelines, two sets of runbooks. Flink's SQL and Table API have matured significantly. A Flink-only architecture would reduce operational surface area substantially and is worth benchmarking before committing to a two-engine design.

 - 03 **Specify ML circuit-breaker behaviour before writing happy-path code.** The fallback to rule-only scoring when the ML service degrades was added reactively after a staging environment incident. Any external

dependency on the critical latency path needs its degraded-mode behaviour specified as a requirement before implementation begins.

- 04 **Schema registry from day one.** Kafka schemas evolved across the project lifecycle. We ended up with a backward-compatibility shim that functioned but created fragility. Enforcing Avro schemas through Confluent Schema Registry at the producer boundary from the start would have eliminated an entire class of production surprises.

Conclusion

The shift from batch to streaming fraud detection is not primarily a latency improvement — it is a change in the intervention model. Batch systems are forensic: they identify what happened. Streaming systems are preventive: they enable action before harm occurs. For a company processing high-volume international money movement, this distinction has direct business and regulatory consequences.

The architecture described here relies on mature, well-understood components: Kafka, Flink, and Spark. The engineering challenge is not in the individual tools but in the compositional decisions — how they are connected, where the ownership boundaries sit, and what constraints each stage is held to. Getting those decisions right, and being honest about the ones that could be improved, is where the real work lies.